LLM Inference From First Principles: Tokens, Logits and the KV Cache

Most of us meet LLMs through one line of code: `model.generate(...)`. It works, and it hides almost everything interesting. In this article I rebuild the core of LLM inference by hand using a small pretrained model, Qwen2.5-0.5B-Instruct, on a free Colab T4 GPU. The goal is not to benchmark anything. It is to see, with real tensors and real numbers, what an inference runtime actually does, and why memory (not math) turns out to be the central problem in serving LLMs.

By the end, we will have gone from raw text to a working token-by-token generation loop, opened up the KV cache, and calculated exactly how much GPU memory each token costs.

1. The setup

We need three things: a GPU PyTorch can see, Hugging Face Transformers, and the model's tokenizer.

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```

On Colab this prints `True` and `Tesla T4`. The stack underneath is simple: Python calls PyTorch, PyTorch launches CUDA kernels, and CUDA runs them on the T4.

Why Hugging Face? Because we don't want to train a model. We want pretrained weights, architecture and tokenizer, and then we handle the runtime logic ourselves: prefill, decode, the KV cache, sampling, request state and scheduling.

2. Text becomes token IDs

A model never sees text. A tokenizer converts raw text into pieces from a learned vocabulary, then into integer IDs.

```
from transformers import AutoTokenizer
model_name = "Qwen/Qwen2.5-0.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_name)

prompt = "capital of france is paris"
inputs = tokenizer(prompt, return_tensors="pt")
print(inputs)
```

The result:

```
{'input_ids': tensor([[65063, 315, 47587, 374, 40858]]),
 'attention_mask': tensor([[1, 1, 1, 1, 1]])}
```

Decoding each ID individually shows how Qwen split the sentence:

```
65063 'capital'
315   ' of'
47587 ' france'
374   ' is'
40858 ' paris'
```

Two things are worth noticing. First, the space is part of the token (`' of'`, not `'of'`), so tokenization is not the same as splitting on spaces. Second, these integers carry no meaning by themselves. `65063` does not mathematically mean "capital"; it is just an index into the vocabulary. The tokenizer and the model must agree on that vocabulary, which is why `AutoTokenizer` loads the tokenizer that belongs to the chosen model.

The **attention mask** here is all ones, meaning every position is a real token. Its job is to mark padding (`0`) when several prompts of different lengths are batched into one rectangular tensor. It is not the causal mask that stops a token from looking at the future; that comes later.

3. Loading the model

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    model_name, dtype=torch.float16
).cuda()
model.eval()
```

A few details matter here:

- **Causal LM** means the model predicts the next token using only the tokens before it.
- `from_pretrained` builds the architecture and loads Qwen's already-trained weights (about 988 MB in `safetensors` format for this model).
- `float16` uses 2 bytes per number, halving memory and memory traffic compared with float32. For inference that matters enormously.
- `.cuda()` moves the weights into GPU memory.
- `model.eval()` switches off training-only behavior like dropout. It does *not* disable gradient tracking; that is `torch.no_grad()`, which we use at run time.

Printing the model reveals its structure, and a few lines of it will pay off later:

- `Embedding(151936, 896)`: a vocabulary of 151,936 tokens, each represented by 896 numbers.
- `24 x Qwen2DecoderLayer`: 24 stacked decoder blocks (the original 2017 Transformer used 6 decoder layers at width 512).
- `q_proj: 896 → 896` but `k_proj` and `v_proj: 896 → 128`. This asymmetry is Grouped Query Attention, and we will come back to it.
- `lm_head: 896 → 151936`, which turns a hidden state into one score per vocabulary token.

4. Embeddings are just a row lookup

```
input_ids = inputs["input_ids"].cuda()
embeddings = model.model.embed_tokens(input_ids)
print(embeddings.shape)   # torch.Size([1, 5, 896])
```

The shape reads `[batch, tokens, hidden size]`: one prompt, five tokens, 896 numbers each. We can prove the embedding is nothing more than a table lookup:

```
token_id = inputs["input_ids"][0, 0].item()   # 65063
print(torch.equal(
    embeddings[0, 0],
    model.model.embed_tokens.weight[token_id]
))  # True
```

Token `65063` simply fetches row `65063` of a `[151936, 896]` weight matrix. Since those are learned weights, that matrix alone holds roughly 136 million parameters (151,936 × 896) before we count attention, MLPs or normalization.

5. The first full forward pass

```
with torch.no_grad():
    outputs = model(input_ids)

print(outputs.keys())  # ['logits', 'past_key_values']
print(outputs.logits.shape)  # [1, 5, 151936]
```

`torch.no_grad()` tells PyTorch we are only running inference, so it skips the bookkeeping that training needs for gradients.

The model returns two things that define everything that follows:

- **`logits`**: raw, unnormalized scores for what token could come next.
- **`past_key_values`**: the KV cache built while processing the prompt.

The logits have shape `[1, 5, 151936]`: a full vocabulary of scores for *each* of the 5 positions, because a causal language model learns next-token prediction at every position. But for generation we only care about the last one, which answers: "given everything up to *paris*, what comes next?"

```
last_token_logits = outputs.logits[:, -1, :]   # [1, 151936]
```

6. Choosing a token

The simplest strategy is greedy decoding: take the highest score.

```
next_token_id = torch.argmax(last_token_logits, dim=-1)
print(next_token_id)                           # tensor([198])
print(repr(tokenizer.decode(next_token_id)))   # '\n'
```

The winner is token `198`, a newline. That makes sense: the prompt already reads like a finished statement. This is the whole trick hidden inside `generate()`:

```
text → tokenizer → IDs → model → 151,936 logits → argmax → ID → decode → text
```

Use `repr()` when debugging tokens; otherwise a newline is invisible.

Greedy decoding always picks the winner, even if the runner-up is nearly as likely. Real systems often sample from the probability distribution instead, which is where we are headed next.

7. Prefill vs decode and the KV cache

We have one new token. To get the next one, the naive approach re-runs the entire sequence through the model. That repeats a lot of work. Instead, we reuse the cache.

```
cache = outputs.past_key_values
print(cache.get_seq_length())   # 5
```

The cache is a `DynamicCache` with 24 layers, one per Transformer layer, because each layer computes its own attention and needs its own stored Keys and Values. It currently holds 5 tokens, the prompt we just processed.

This gives us the two phases at the heart of LLM serving:

- **Prefill**: process the whole prompt at once (`[1, 5]`), build the cache, predict the first token.
- **Decode**: feed only the newest token (`[1, 1]`) plus the existing cache, extend the cache by one entry, predict the next token.

```
with torch.no_grad():
    decode_output = model(
        next_token_id.unsqueeze(0),   # [1] -> [1, 1]
        past_key_values=cache,
        use_cache=True,
    )

print(decode_output.logits.shape)                    # [1, 1, 151936]
print(decode_output.past_key_values.get_seq_length())  # 6
```

`unsqueeze(0)` restores the batch dimension the model expects. Notice the logits shape: prefill gave `[1, 5, 151936]`, decode gives `[1, 1, 151936]`, because we only processed one token. The cache grew from 5 to 6.

One subtle point: a token is only added to the cache when it has actually been *run through the model*. After we predict `"Paris"` as the second token, the cache still has length 6, because `"Paris"` has been chosen but not yet processed. The chosen token is always

one step ahead of the cache.

Also remember that `DynamicCache` is mutable. The model updates it in place, so re-running a decode cell by accident silently grows the cache again. Cache state is real state.

8. The generation loop

After resetting to a fresh prefill, the whole generation process fits in a few lines:

```
generated_ids = []

for _ in range(5):
    generated_ids.append(next_token_id.item())

    with torch.no_grad():
        outputs = model(
            input_ids=next_token_id.unsqueeze(0),
            past_key_values=cache,
            use_cache=True,
        )

    next_token_id = torch.argmax(outputs.logits[:, -1, :], dim=-1)

print(tokenizer.decode(generated_ids))
```

The output is `\nParis, the capital` (with a leading newline that is easy to miss).

The loop's logic, step by step: store the token already predicted, process it through the model (which adds its K/V to the cache and yields new logits), then replace `next_token_id` with the new prediction. Nothing is "shifted" numerically; the variable is simply overwritten by the next prediction. After the final iteration the model has predicted a sixth token that we never store, so `generated_ids` holds exactly five.

It also shows two distinct pieces of state that serve different purposes: `generated_ids` is what we return to the user, while the KV cache is what the model needs to keep going.

A production engine adds batching, scheduling, memory management, stopping conditions and optimized kernels, but this dependency never changes: predict a token, feed it back in, predict the next one.

9. Opening up the KV cache

After the loop, the cache holds 10 tokens (5 prompt + 5 generated). Looking at layer 0:

```
layer0 = cache.layers[0]
print(layer0.keys.shape)    # [1, 2, 10, 64]
print(layer0.values.shape)  # [1, 2, 10, 64]
```

The shape is `[batch, KV heads, cached tokens, head dimension]`. The values inside are intermediate vectors produced by `k_proj` and `v_proj`; we don't interpret them, we just store them so old tokens never need recomputing.

Why only 2 KV heads?

```
Query heads: 14
KV heads:    2
Hidden size: 896
Head dim:    64   (896 / 14)
```

This is **Grouped Query Attention (GQA)**. Instead of 14 Key and 14 Value heads, Qwen has 14 Query heads sharing just 2 K/V heads, with several query heads reading from the same one. The earlier architecture printout already hinted at it:

- `q_proj`: 14 heads × 64 = 896
- `k_proj`: 2 heads × 64 = 128
- `v_proj`: 2 heads × 64 = 128

The motivation is purely about inference. Query vectors are needed only for the token currently being processed, but K and V for *every* previous token must stay in memory for the entire generation. Fewer KV heads means a smaller cache, less GPU memory used, and less data moved on every decode step.

10. Counting the memory

For one layer: `1 × 2 × 10 × 64 = 1,280` elements for K and 1,280 for V, at 2 bytes each (float16), so 5,120 bytes per layer. Summing across all 24 layers:

```
Total KV cache: 122,880 bytes = 120 KB   (10 tokens)
Per token:       12,288 bytes =  12 KB
```

That number falls straight out of the architecture:

```
KV bytes per token = layers × KV heads × head_dim × 2 (K and V) × bytes per
value
                   = 24 × 2 × 64 × 2 × 2
                   = 12,288 bytes
```

This formula is worth memorizing. It tells you what any model's context will cost before you run a single request.

11. Why this becomes a serving problem

At 12 KB per token, this tiny model looks harmless, but cache memory scales linearly with every token of every active request:

Workload	KV cache (approx.)
1 request × 1,000 tokens	12 MB
1 request × 10,000 tokens	120 MB
100 requests × 1,000 tokens	1.2 GB
100 requests × 10,000 tokens	12 GB

And this is a 0.5B model. Larger models have far bigger per-token footprints. The weights stay roughly fixed in GPU memory, but the KV cache fluctuates constantly with the number of active requests times the tokens each has cached. That moving target is why serving engines such as vLLM obsess over KV cache allocation, block management, paged KV caches, prefix caching, eviction, preemption and fragmentation.

12. One more detail: who decides when to stop?

Qwen's end-of-sequence token is `<|im_end|>` (ID `151645`). It is just another token in the vocabulary. The model does not terminate your Python loop; it merely predicts the EOS token, and the *runtime* decides to stop that request. This division of labor matters a lot once many requests are scheduled together, because each one finishes at a different time.

What we built

Starting from `generate()` as a black box, we now understand and have manually reproduced:

1. Text to token IDs, and why IDs carry no meaning on their own.
2. Token IDs to embeddings, which is a plain row lookup in a learned matrix.
3. A forward pass producing logits over the whole vocabulary at every position.
4. Greedy token selection with `argmax`.
5. Prefill and decode, and how the KV cache lets decode process one token at a time.
6. A complete autoregressive loop.
7. The structure of the cache, how GQA shrinks it, and the formula for its memory cost.

The next step is to replace `argmax` with softmax and sampling, so the model can choose among likely tokens instead of always taking the winner. That is where temperature, top-k and top-p come in.